



RELAZIONE CARRY-SELECT 16 BIT UNSIGNED

A CURA DI:

MARIAIDA PERRI – MAT.239943

ANDREEA ROXANA MANOLACHE – MAT.245906

ANDREA MAZZA – MAT.240059

FRANCESCO PIO RUFFO – MAT.240044

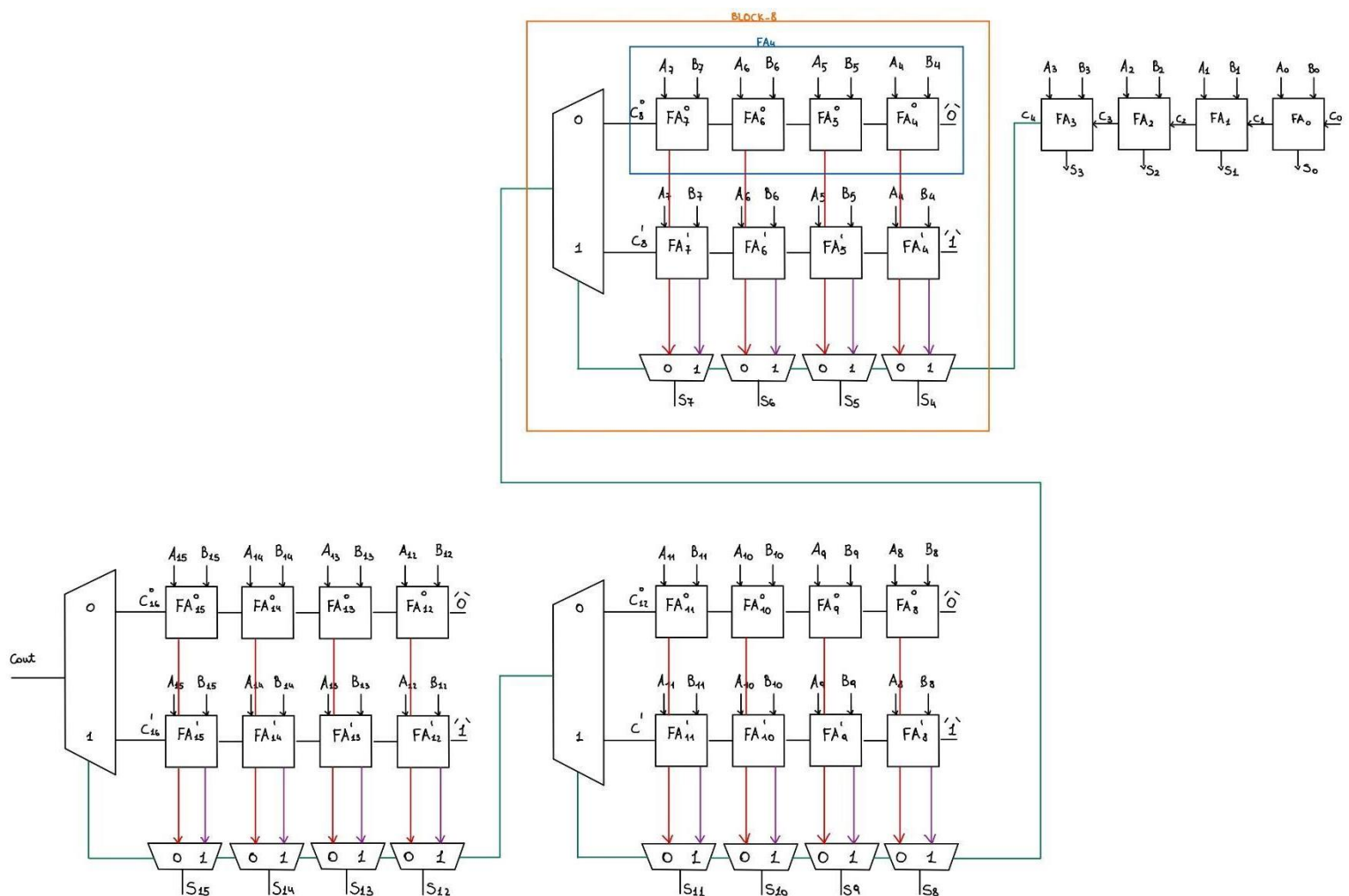
A.A. 2024/25

ARCHITETTURA DI RIFERIMENTO E SCELTE PROGETTUALI

Per la realizzazione del carry-select a 16 bit, si è utilizzato un approccio descrittivo di tipo gerarchico, consistente nell'utilizzo di più sottostrutture, che verranno analizzate nel prosieguo.

I principali vantaggi di tale scelta sono:

- **Modularità del sistema:** ciascuna sottocomponente assolve uno specifico ruolo ed è capace di interagire con le altre componenti. La scomposizione del problema principale in sottoproblemi segue un approccio simil *Divide et Impera*.
- **Manutenibilità del codice:** consente di apportare modifiche ai singoli componenti senza compromettere il resto del sistema.
- **Scalabilità del sistema:** rende semplice l'aggiunta di risorse al sistema, riutilizzando le componenti già impiegate.



CENNI SU VHDL

Il linguaggio VHDL (*Very High Speed Integrated Circuit Hardware Description Language*) è usato per la **descrizione** dell'hardware; non è un linguaggio eseguibile, ma descrive le componenti di un circuito digitale.

Inoltre, è possibile simulare una specifica in modo da verificare il comportamento del circuito in relazione ai diversi input forniti.

VHDL consente inoltre di fornire specifiche di circuiti digitali a più livelli di astrazione.

A livello più basso, è possibile rappresentare gli elementi logici di un circuito, semplici o complessi, specificando le loro interconnessioni.

La progettazione consta di due fasi:

- Descrizione dell'oggetto come **Entity**: si elencano le porte di comunicazione in input e in output;
- Progettazione della **Architecture**: rappresenta l'architettura interna in cui viene esplicitato il funzionamento del circuito, specificando la descrizione funzionale vera e propria.

L'implementazione della Architecture può avvenire seguendo due diverse modalità:

- **Behavioral**: ogni modulo viene descritto in base a come si comporta, piuttosto che determinando quali sono i suoi componenti e come sono interconnessi; viene specificata dunque la relazione che sussiste tra ingressi e uscite;
- **Structural**: ogni modulo è descritto come una collezione di porte e componenti più a basso livello, interconnessi per realizzare un'opportuna funzione.

FULL-ADDER (FA)

Il **FULL-ADDER** (o sommatore completo), è un circuito logico progettato per eseguire l'operazione di somma tra tre bit in ingresso: i due bit costituenti i bit effettivi della somma (A e B) e un terzo bit in ingresso denominato C_{in} (carry-in o riporto in ingresso). Le uscite del circuito sono due, e comprendono un bit di somma (S) e un bit di riporto (C_{out}), che rappresenta il riporto della somma dei due bit in ingresso.

La struttura del Full-Adder costituisce la base per i circuiti sommatore più complessi, infatti, è possibile collegare più circuiti di questo tipo in modo da poter sommare numeri di più bit.

Tabella di verità

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Mappe di Karnaugh

S:

X \ YZ	00	01	11	10
	0	1		1
1	1		1	

C_{out} :

X \ YZ	00	01	11	10
	0		1	
1		1	1	1

Equazioni logiche

$$S = A \oplus B \oplus C_{in} = (A \oplus B) \oplus C_{in}$$

$$C_{out} = A B + A C_{in} + B C_{in} = A B + C_{in}(A + B)$$

Si può notare inoltre come C_{out} ha valore 1 se $B = A = 1$ o se $A \neq B$ e $C_{in} = 1$.

Quindi è possibile scrivere:

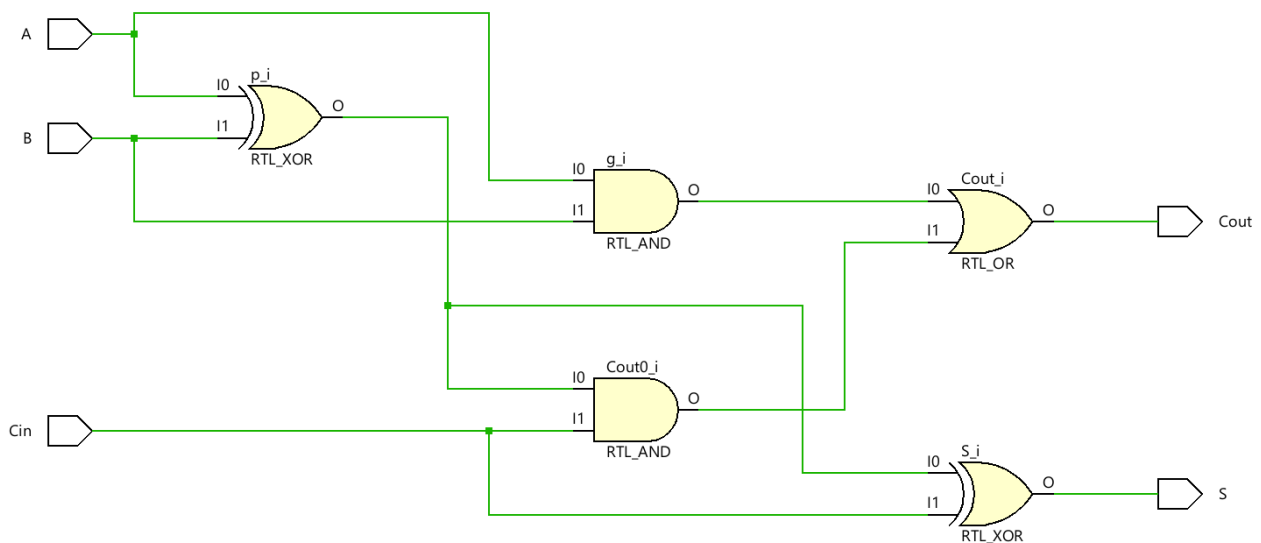
$$A B + C_{in}(A \oplus B)$$

Si introducono in questo modo due segnali, il segnale **Propagate** e il segnale **Generate**:

$$P = A \oplus B$$

$$G = A B$$

È possibile visualizzare tramite Vivado lo *schematic* di un FA:



L'implementazione in VHDL è la seguente:

```
1 entity FA is
2     Port ( A : in bit;
3           B : in bit;
4           Cin : in bit;
5           Cout : out bit;
6           S : out bit);
7 end FA;
8
9 architecture ArcFA of FA is
10     signal p,g : bit;
11
12     begin
13         p <= A xor B;
14         s <= p xor Cin;
15         Cout <= g or (p and Cin);
16         g <= A and B;
17     end ArcFA;
18
```

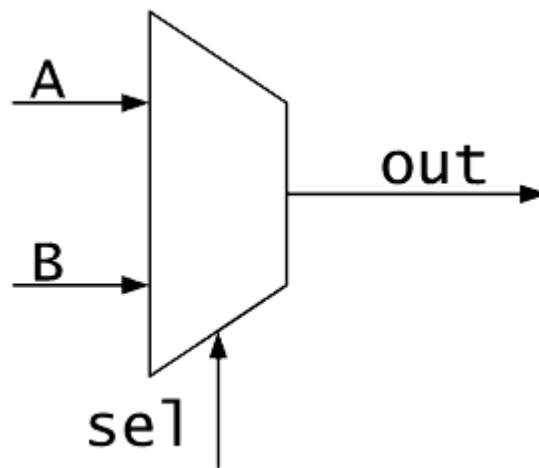
MULTIPLEXER (MUX)

Il **MULTIPLEXER** è un circuito in grado di selezionare uno fra 2^n ingressi e di instradare il dato in esso contenuto in output: per questo motivo, oltre agli input forniti, vengono impiegati anche m bit di selezione (detti appunto “selettori”).

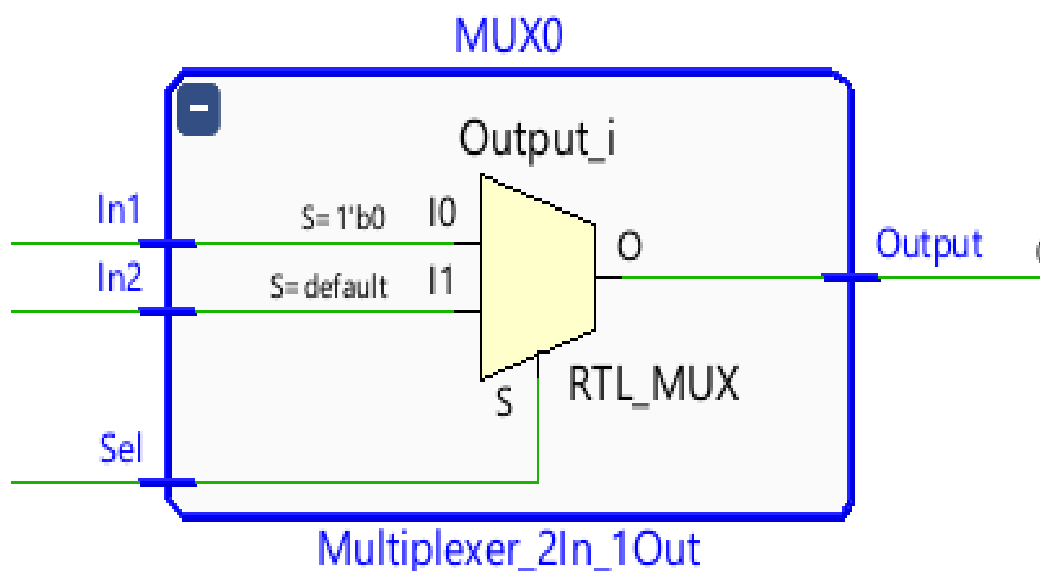
Tra gli n ingressi forniti e gli m ingressi di selezione sussiste la seguente relazione:

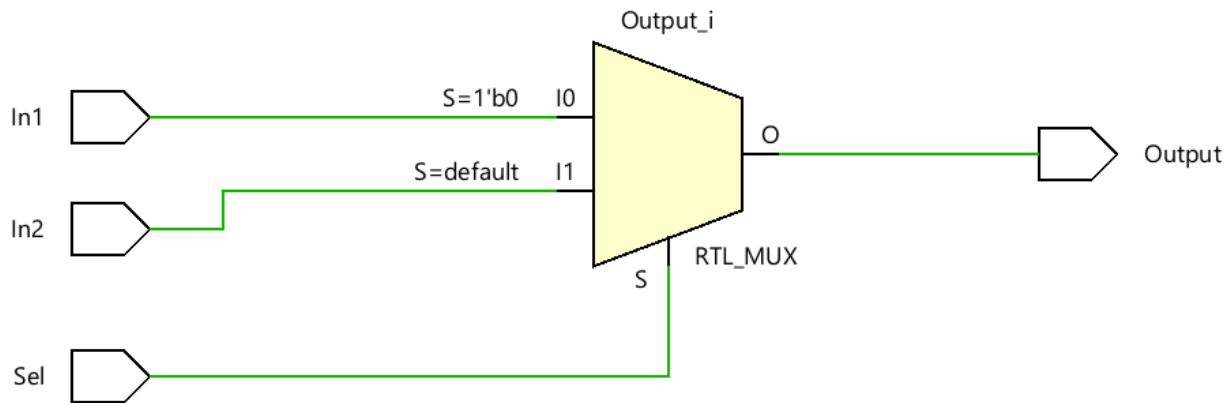
$$n = 2^m \Leftrightarrow m = \log_2 n$$

Descrivendo dunque un multiplexer con 2 soli ingressi dati, sarà necessaria 1 sola linea di selezione, dal momento che si hanno 2^2 possibili configurazioni binarie.



Mediante gli strumenti messi a disposizione da Vivado, è possibile riportare lo schematic del Multiplexer:





Il Mux rappresenta una componente fondamentale del Carry Select, poiché consente di selezionare una delle due somme parziali calcolate in parallelo: in base al valore del riporto del blocco precedente (che viene usato come selettore) seleziona il risultato corretto tra i due possibili.

La selezione avviene rapidamente e può essere dunque sfruttata per parallelizzare al meglio le operazioni di somma.

L'implementazione in VHDL viene scritta come:

```

1  entity Multiplexer_2In_1Out is
2      Port (In1: in bit;
3            In2 : in bit;
4            Sel: in bit;
5            Output : out bit);
6  end Multiplexer_2In_1Out;
7
8
9
10 architecture Behavioral of Multiplexer_2In_1Out is
11
12     begin
13         Output <= In1 when Sel = '0' else In2;
14
15     end Behavioral;
16
17

```

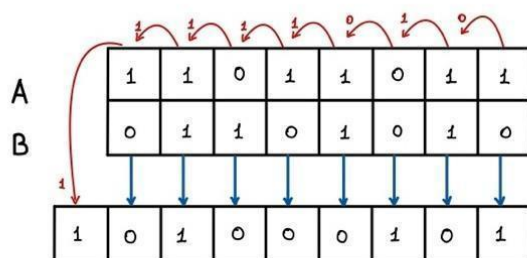
Come emerge dalla porzione di codice riportata, la descrizione del Mux è estremamente breve grazie all'utilizzo del costrutto **when-else**, che replica perfettamente la “logica” del circuito considerato.

RIPPLE-CARRY ADDER (RCA)

Il **RIPPLE-CARRY ADDER (RCA)**, detto anche *addizionatore a propagazione di riporto*, è il metodo più diretto per realizzare un addizionatore ad n bit. È un circuito che richiede n Full-Adder in cascata, cioè con il riporto uscente dall' i -esimo Full-Adder collegato al riporto entrante dell' $(i+1)$ -esimo Full-Adder.

Il circuito RCA calcola la somma così come verrebbe calcolata a mano.

Si consideri il seguente esempio:

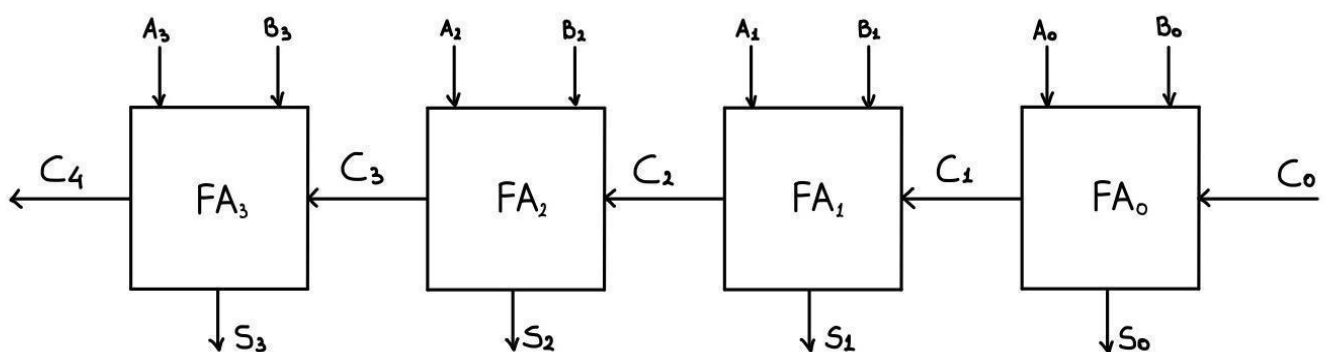


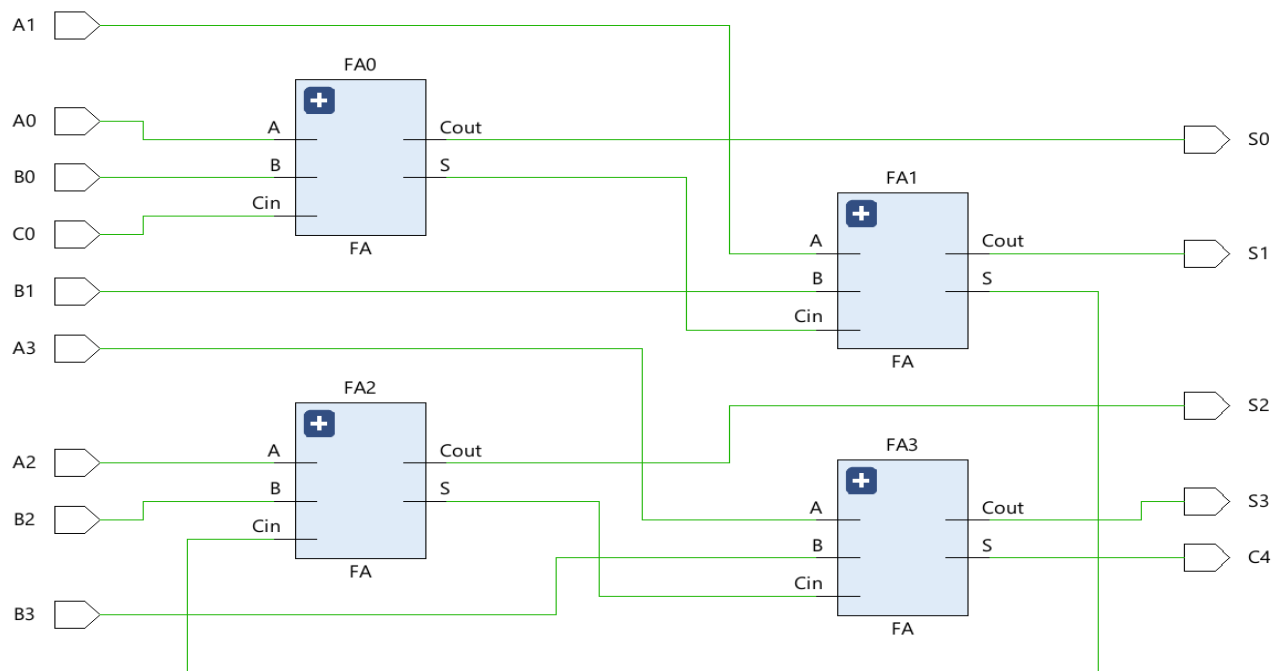
L'operazione di somma viene effettuata procedendo dalla coppia di cifre meno significative, più a destra, verso la coppia di cifre più significative, a sinistra.

Ogni coppia di cifre è sommata insieme ad un eventuale riporto generato dalla coppia di cifre a destra,

producendo a sua volta un bit di somma ed un eventuale bit di riporto che sarà tenuto in conto nella somma successiva.

Per la realizzazione del Carry-Select a 16 bit si è fatto uso di RCA a 4 bit, costituiti da 4 Full-Adder collegati in cascata.





L'architettura del circuito è abbastanza semplice, tuttavia, può risultare molto lenta; nel caso peggiore dipende linearmente dal numero degli stadi, in quanto bisogna aspettare che il riporto si propaghi dalla prima all'ultima cella per ottenere il risultato corretto.

L'RCA a 4 bit è stato implementato in VHDL nel seguente modo:

```

1 entity FA4 is
2     port ( C0,A0,A1,A2,A3,B0,B1,B2,B3 : in bit;
3           C4,S3,S2,S1,S0 : out bit);
4 end FA4;
5
6 architecture ArcFA4 of FA4 is
7
8     signal C1,C2,C3 : bit;
9
10    component FA
11    port(A,B,Cin : in bit;
12         Cout,S : out bit);
13 end component;
14
15    begin
16        FA0: FA port map(A0,B0,C0,C1,S0);
17        FA1: FA port map(A1,B1,C1,C2,S1);
18        FA2: FA port map(A2,B2,C2,C3,S2);
19        FA3: FA port map(A3,B3,C3,C4,S3);
20    end ArcFA4;
21
22

```

Come si può notare, esso richiama il component FA, corrispondente al Full-Adder, che viene istanziato 4 volte.

BLOCCO DA 8 FA

Il blocco contenente 8 full-adder è costituito da **due ripple-carry**, con 4 full-adder ciascuno. Tale struttura rappresenta il blocco costitutivo di base per un carry-select.

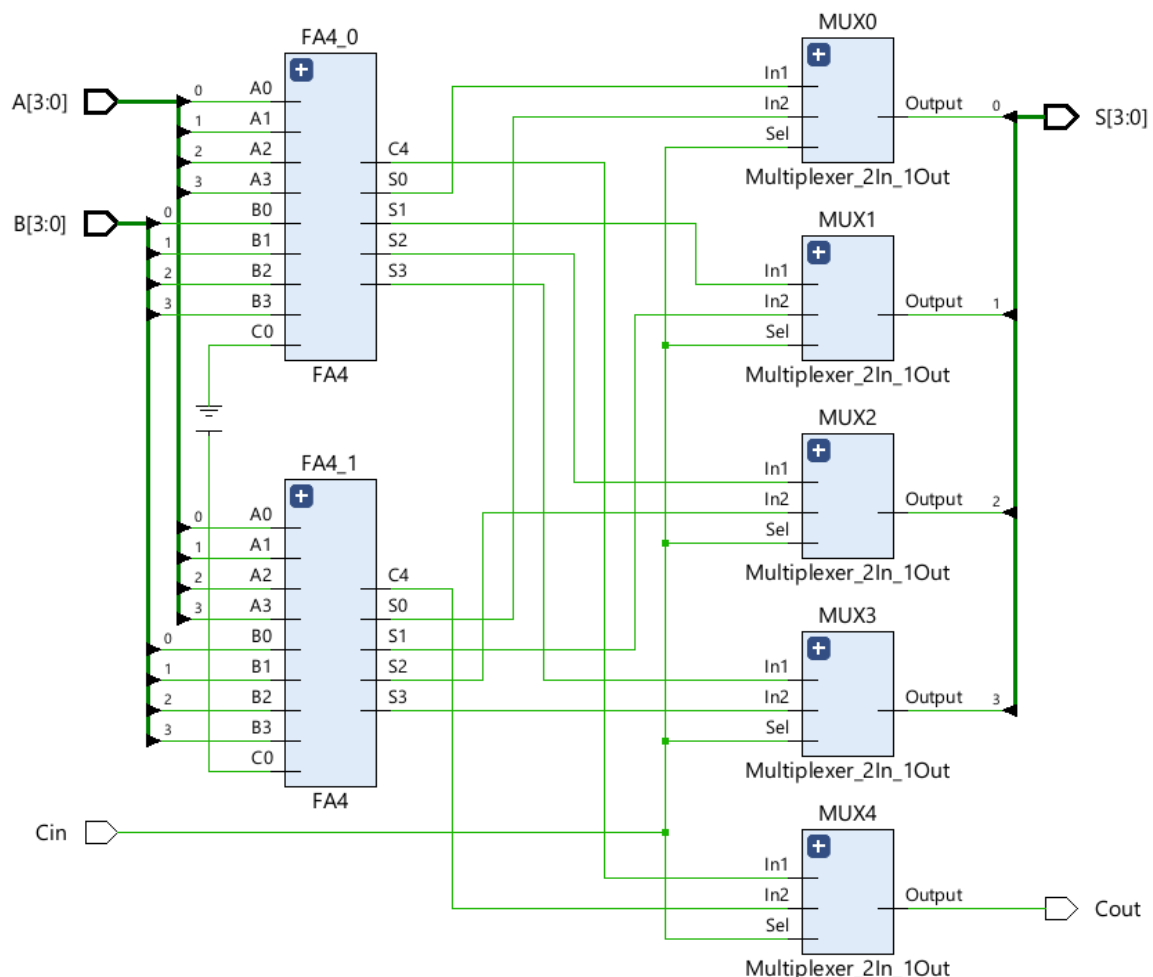
Ogni FA esegue la somma dei due bit in ingresso, propagando il riporto al FA successivo. L'utilizzo di due ripple-carry consente di eseguire il calcolo nell'ipotesi in cui il riporto in ingresso sia 0 e nell'ipotesi in cui sia 1. I risultati della somma calcolati dagli 8 FA sono collegati a 4 multiplexer, che selezionano il risultato corretto, tenendo conto del riporto effettivo in ingresso, pari a 0 o 1.

Inoltre, i riporti in uscita dei due ripple-carry ed il riporto C_{in} , che rappresentano rispettivamente i due ingressi e il segnale di selezione del multiplexer, consentono a quest'ultimo di condurre il riporto in uscita dal blocco.

Grazie a questa configurazione, un carry-select riduce il tempo di somma di due numeri rispetto ad un ripple-carry, in quanto calcola parallelamente il risultato, sia per un riporto in ingresso uguale a 0, sia per un riporto in ingresso uguale a 1.

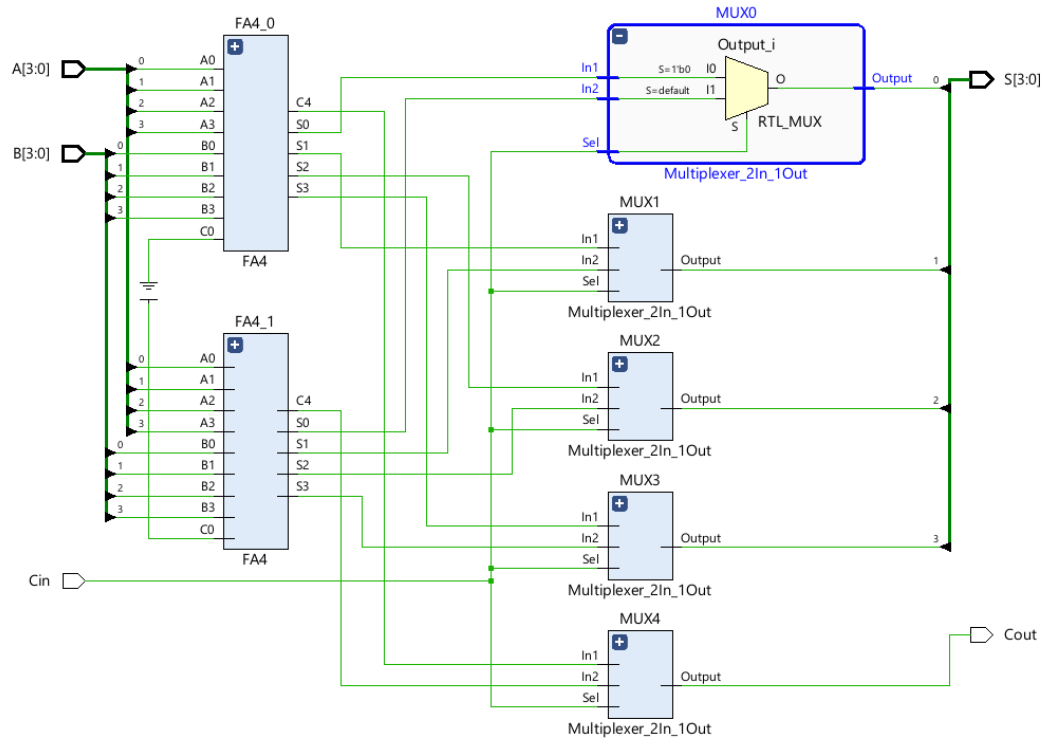
Tuttavia, il circuito diventa più costoso per numero di porte e collegamenti, poiché un circuito con più porte dissipa più energia.

Per un carry-select a 16 bit, sono stati impiegati 3 blocchi di questo tipo, più un blocco iniziale costituito da un ripple-carry a 4 bit.

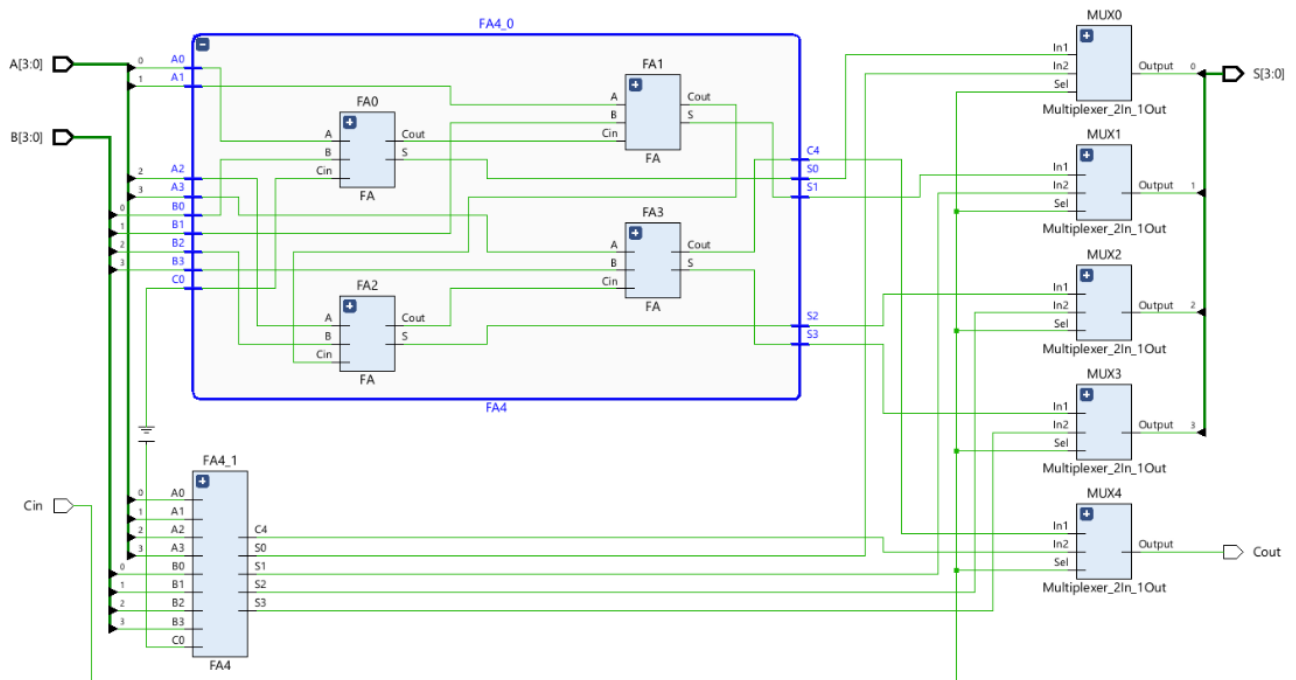


Naturalmente, dal precedente *schematic* è possibile risalire alla struttura delle sottocomponenti, focalizzandosi su:

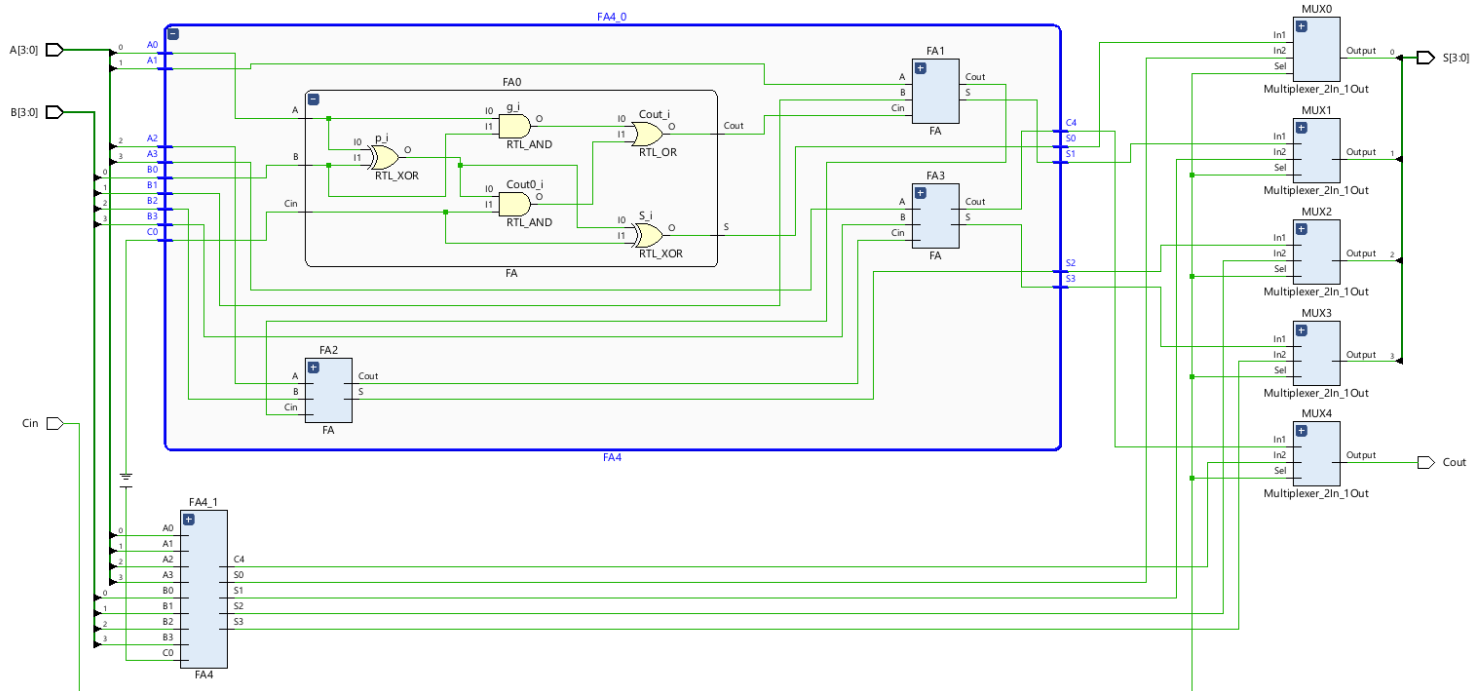
- **Multiplexer**



- **Ripple-carry**



- Full-adder



In VHDL, il blocco costitutivo di base del carry-select si implementa nel seguente modo:

```
1 entity block_8 is
2 port (
3     A,B : in bit_vector(3 downto 0);    --bit in input
4     Cin : in bit;    --riporto per selettore mux
5     Cout : out bit;    --riporto ultimo mux
6     S : out bit_vector(3 downto 0)); --vettore delle uscite dei multiplexer
7 end block_8;
8
9 architecture Behavioral of block_8 is
10
11     signal sumFA0 : bit_vector(3 downto 0);
12     signal sumFA1 : bit_vector(3 downto 0);
13     signal C4_0, C4_1 : bit;
14
15     component FA4
16     port(C0,A0,A1,A2,A3,B0,B1,B2,B3 : in bit;
17         C4,S3,S2,S1,S0 : out bit);
18 end component;
19
20 component Multiplexer_2In_1Out
21 port ( In1: in bit;
22         In2 : in bit;
23         Sel: in bit;
24         Output : out bit);
25 end component;
26
27 begin
28
29     FA4_0 : FA4 port map('0',A(0),A(1),A(2),A(3),B(0),B(1),B(2),B(3),C4_0,sumFA0(3),sumFA0(2),sumFA0(1),sumFA0(0));
30     FA4_1 : FA4 port map('1',A(0),A(1),A(2),A(3),B(0),B(1),B(2),B(3),C4_1,sumFA1(3),sumFA1(2),sumFA1(1),sumFA1(0));
31     MUX0 : Multiplexer_2In_1Out port map(sumFA0(0),sumFA1(0),Cin,S(0));
32     MUX1 : Multiplexer_2In_1Out port map(sumFA0(1),sumFA1(1),Cin,S(1));
33     MUX2 : Multiplexer_2In_1Out port map(sumFA0(2),sumFA1(2),Cin,S(2));
34     MUX3 : Multiplexer_2In_1Out port map(sumFA0(3),sumFA1(3),Cin,S(3));
35
36     MUX4 : Multiplexer_2In_1Out port map (C4_0,C4_1,Cin,Cout);
37
38 end Behavioral;
```

CARRY-SELECT

Un **CARRY-SELECT** è un **circuito sommatore** che permette di velocizzare il calcolo della somma tra due operandi introducendo un certo grado di **ridondanza** nei calcoli.

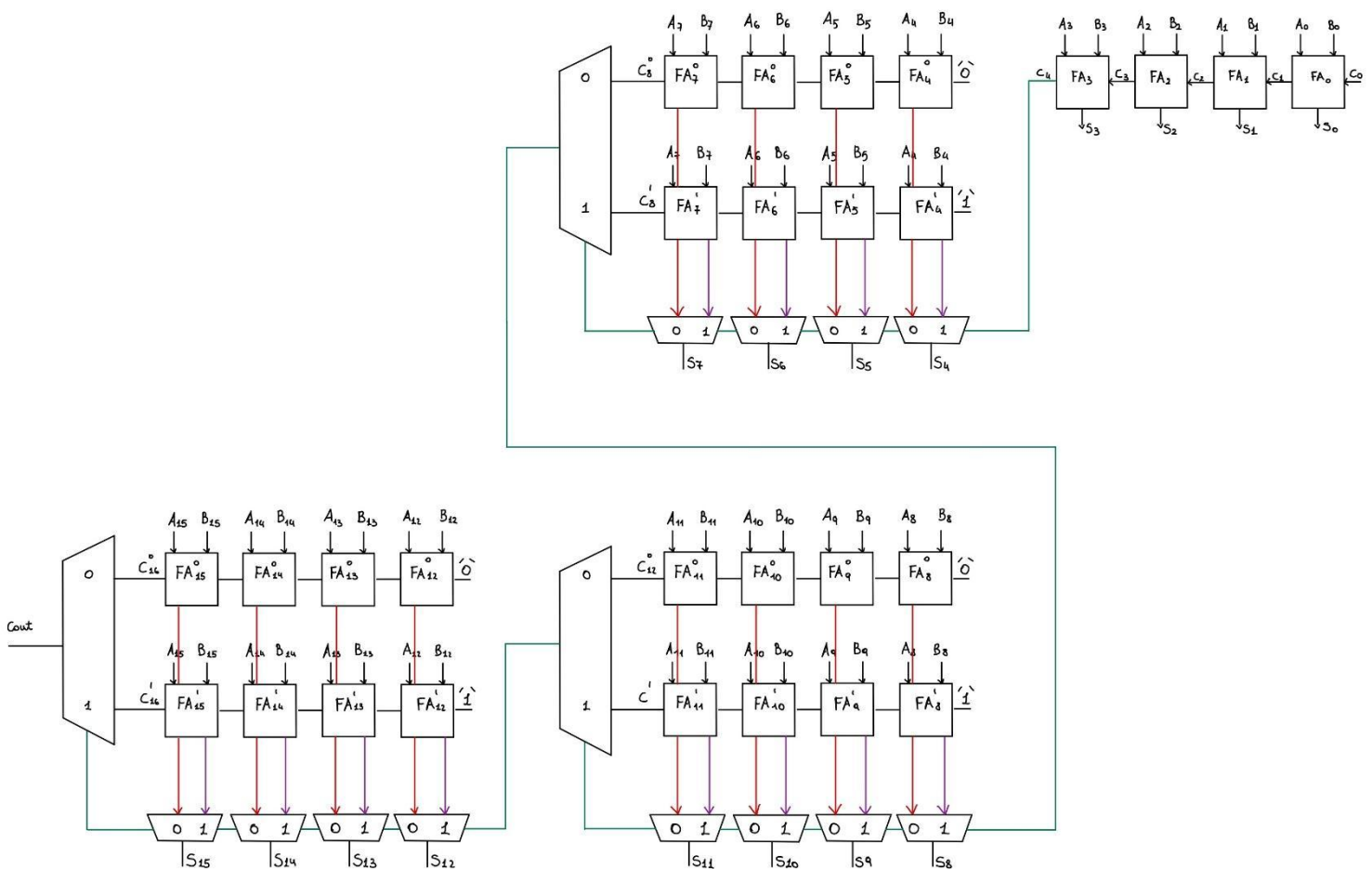
Supponendo di avere due operandi a n bit, l'addizionatore è costituito da p blocchi che elaborano m bit. Per come abbiamo definito queste grandezze, il numero totale di bit dell'addizionatore sarà

$$n = p \times m$$

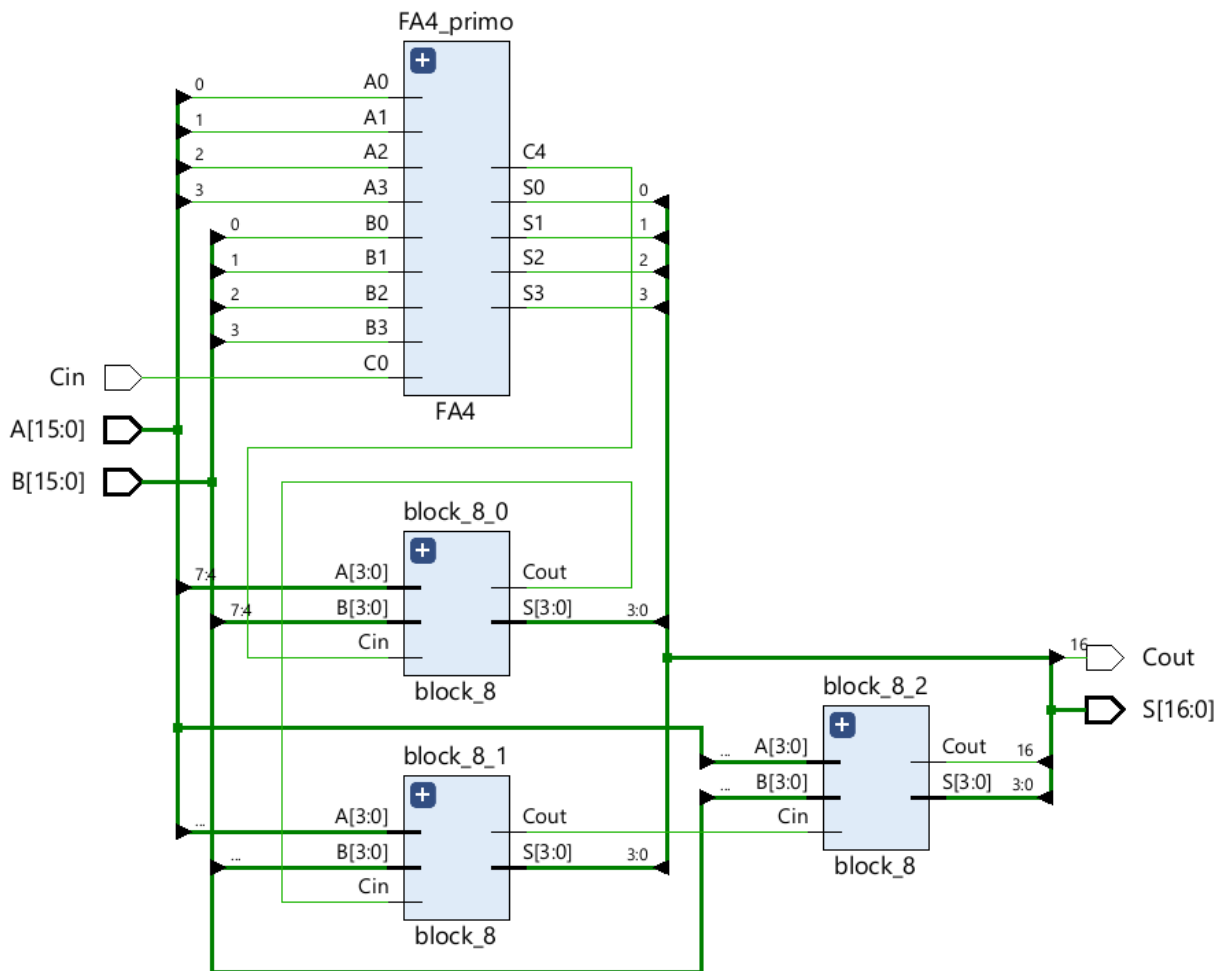
Il primo blocco sarà costituito da un ripple-carry adder (RCA) ad m bit, mentre i blocchi dal secondo al p -esimo saranno costituiti da due RCA ad m bit, uno con il segnale di riporto pari ad 1 e l'altro con il segnale di riporto pari a 0.

Le uscite di questi blocchi saranno selezionate da un multiplexer che riceverà in ingresso come segnale di selezione il riporto uscente dal blocco precedente.

Il parallelismo introdotto all'interno del circuito permette di velocizzare il calcolo. Notiamo, infatti, che i calcoli eseguiti nei diversi blocchi possono procedere indipendentemente. Tuttavia, questo vantaggio comporta un prezzo da pagare in termini di porte logiche usate.



Lo *schematic* corrispondente ad un carry-select a 16 bit è il seguente:



Come ci si aspetta, è costituito da un primo blocco che rappresenta il RCA a 4 bit, e da 3 successivi blocchi contenenti due RCA a 4 bit.

Si è prevista, in fase di progettazione, l'aggiunta di un bit sull'output "S", al fine di gestire eventuali situazioni di overflow.

In VHDL, il carry-select a 16-bit unsigned è implementato come:

```
1 entity carry_select_16_unsigned is
2   port (
3       A,B : in bit_vector(15 downto 0);
4       Cin : in bit;
5       Cout : out bit;
6       S : out bit_vector(16 downto 0));
7 end carry_select_16_unsigned;
8
9 architecture Behavioral of carry_select_16_unsigned is
10
11     signal riporto : bit_vector(3 downto 0);
12
13     component FA4
14     port (C0,A0,A1,A2,A3,B0,B1,B2,B3 : in bit;
15          C4,S3,S2,S1,S0 : out bit);
16 end component;
17
18     component block_8
19     port (
20         A,B : in bit_vector(3 downto 0);    --bit in input
21         Cin : in bit;    --riporto per selettore mux
22         Cout : out bit;  --riporto ultimo mux
23         S : out bit_vector(3 downto 0)); --vettore delle uscite dei multiplexer
24 end component;
25
26 begin
27     FA4_primo : FA4 port map (Cin,A(0),A(1),A(2),A(3),B(0),B(1),B(2),B(3),riporto(0),S(3),S(2),S(1),S(0));
28
29     block_8_0 : block_8 port map (A(7 downto 4),B(7 downto 4),riporto(0),riporto(1),S(7 downto 4));
30     block_8_1 : block_8 port map (A(11 downto 8),B(11 downto 8),riporto(1),riporto(2),S(11 downto 8));
31     block_8_2 : block_8 port map (A(15 downto 12),B(15 downto 12),riporto(2),riporto(3),S(15 downto 12));
32     S(16) <= riporto(3);
33     Cout <= riporto(3);
34
35 end Behavioral;
```

SIMULAZIONE

Tra le varie fasi che compongono il flusso di progetto di un circuito digitale, quella che ricopre un ruolo di fondamentale importanza è sicuramente la **simulazione**.

Per poter verificare che la descrizione del componente segua effettivamente le specifiche progettuali, lo si sottopone a più prove; il compito di ciascuna è quello di andare a testare in modo diverso se utilizzando degli appositi input, i corrispondenti output corrispondano a quelli desiderati.

In particolare, tra le possibili simulazioni, distinguiamo la:

- I. Simulazione logica, anche detta simulazione ideale – si concentra sulla verifica del comportamento logico del circuito;
- II. Simulazione post - sintesi – tiene in considerazione anche le caratteristiche reali, includendo i ritardi associati alle porte logiche. Potrebbero dunque emergere in questa fase dei problemi di natura “elettrica”, non considerati nella precedente fase;
- III. Simulazione post Place & Route – si considerano anche i signals ed i conseguenti ritardi ad essi associati durante la loro propagazione all’interno del circuito
- IV. Hardware Test

TESTBENCH

Si predispone un'apposita area di lavoro, all'interno della quale sarà ovviamente presente la componente che si vuole testare (anche detta **CUT** – Circuit Under Test) e la descrizione dei vari input che si vogliono applicare al CUT.

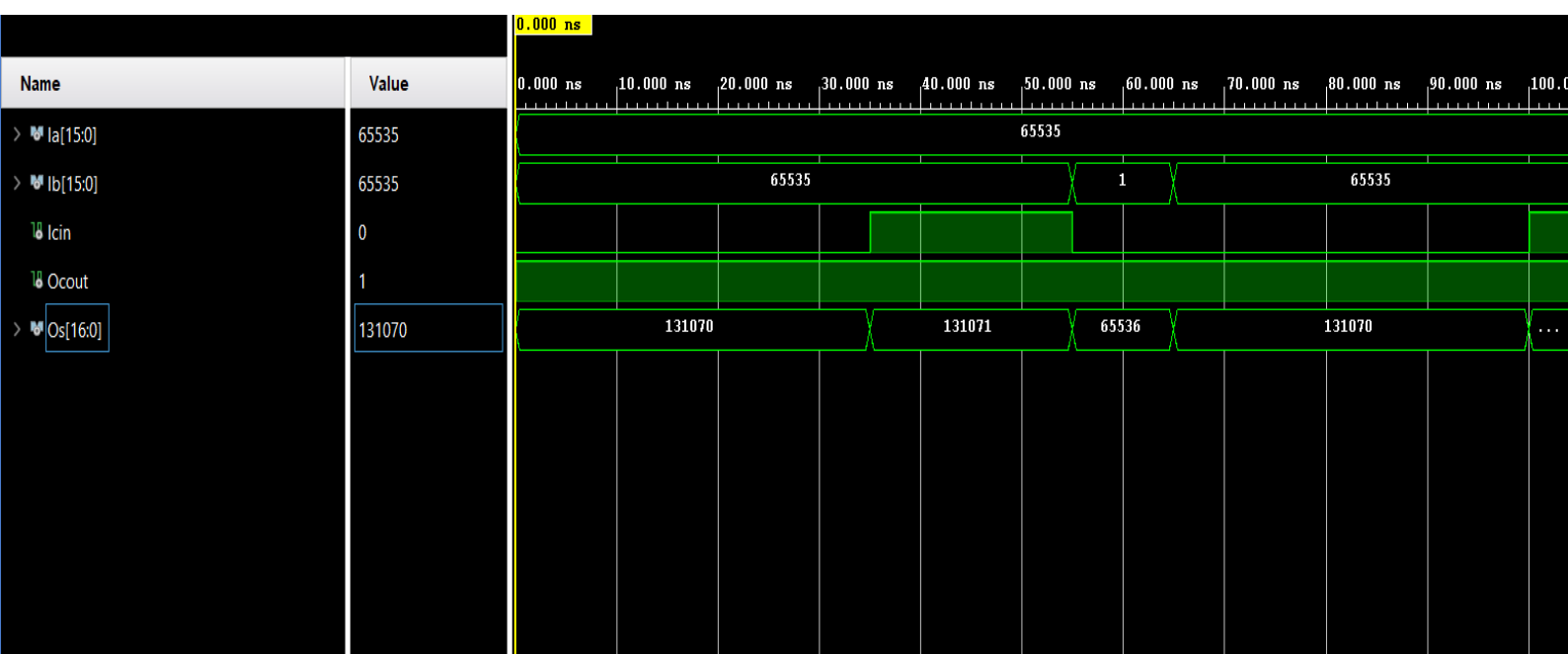
Il testbench non ha una corrispondenza reale sul circuito fisico: per questo motivo la Entity del file VHDL relativo al test sarà vuota.

Per ognuno dei bit/bit_vector in ingresso/uscita viene definito un signal. I signal di ingresso possono essere “pilotati” in modo tale da porre sotto esame situazioni particolari o semplicemente per verificare il comportamento del circuito al variare delle casistiche.

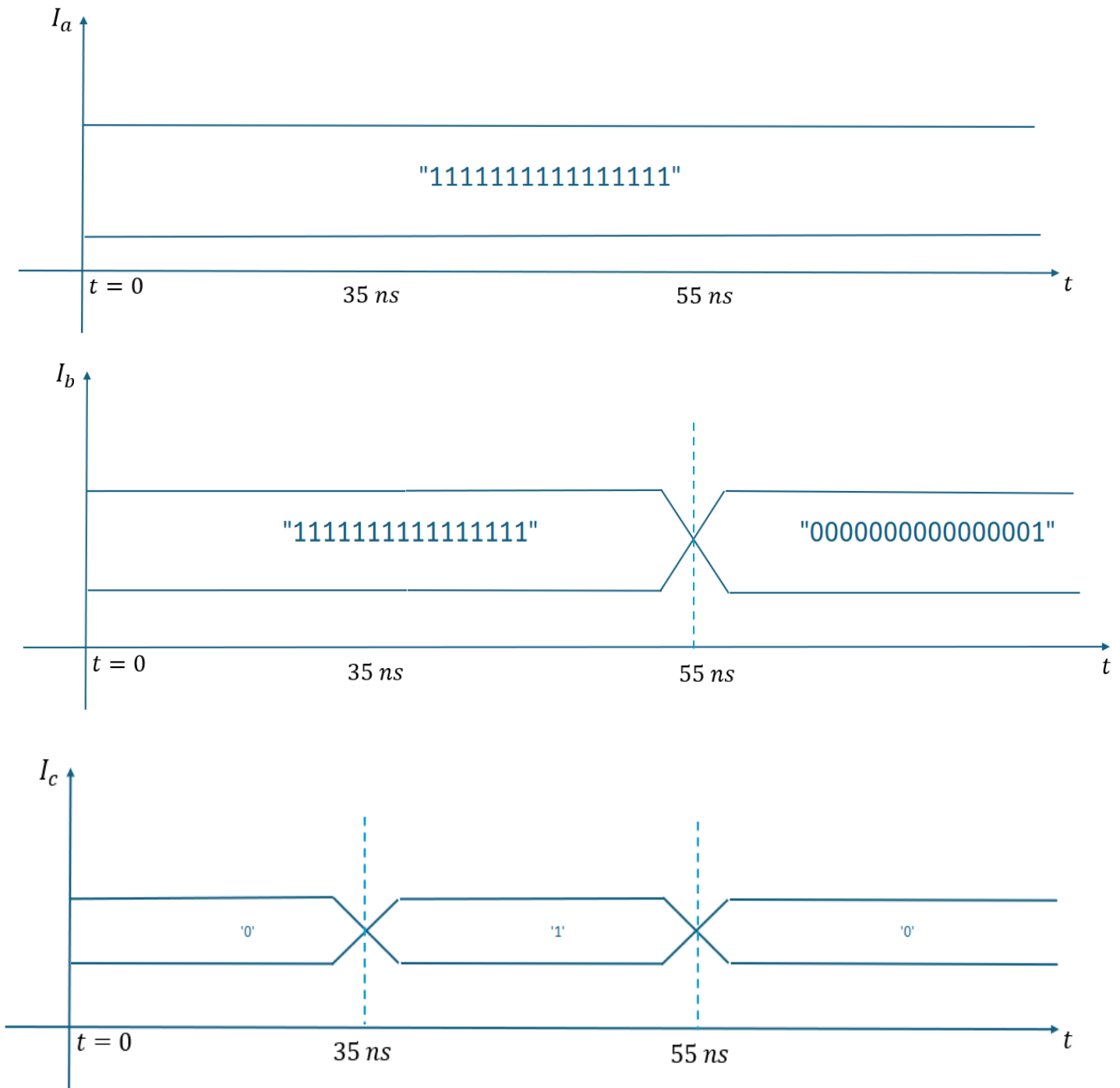
All'interno dell'Architecture dovrà essere poi definito l'inizio e la fine di uno statement (process) all'interno del quale tutte le istruzioni saranno eseguite **sequenzialmente**.

```
1 entity Sim_carry_select_16 is
2 end Sim_carry_select_16;
3
4 architecture Behavioral of Sim_carry_select_16 is
5 component carry_select_16_unsigned
6 port ( A,B : in bit_vector(15 downto 0);
7       Cin : in bit;
8       Cout : out bit;
9       S : out bit_vector(16 downto 0));
10 end component;
11
12 signal Ia,Ib : bit_vector(15 downto 0);
13 signal Icin : bit;
14
15 signal Ocout : bit;
16 signal Os : bit_vector(16 downto 0);
17
18 begin
19     CUT:carry_select_16_unsigned port map (Ia,Ib,Icin,Ocout,Os);
20 process
21 begin
22     --t=0
23     Ia <= "1111111111111111";
24     Ib <= "1111111111111111";
25     Icin <= '0';
26
27     wait for 35 ns;
28     --t=35
29     Ia <= "1111111111111111";
30     Ib <= "1111111111111111";
31     Icin <= '1';
```

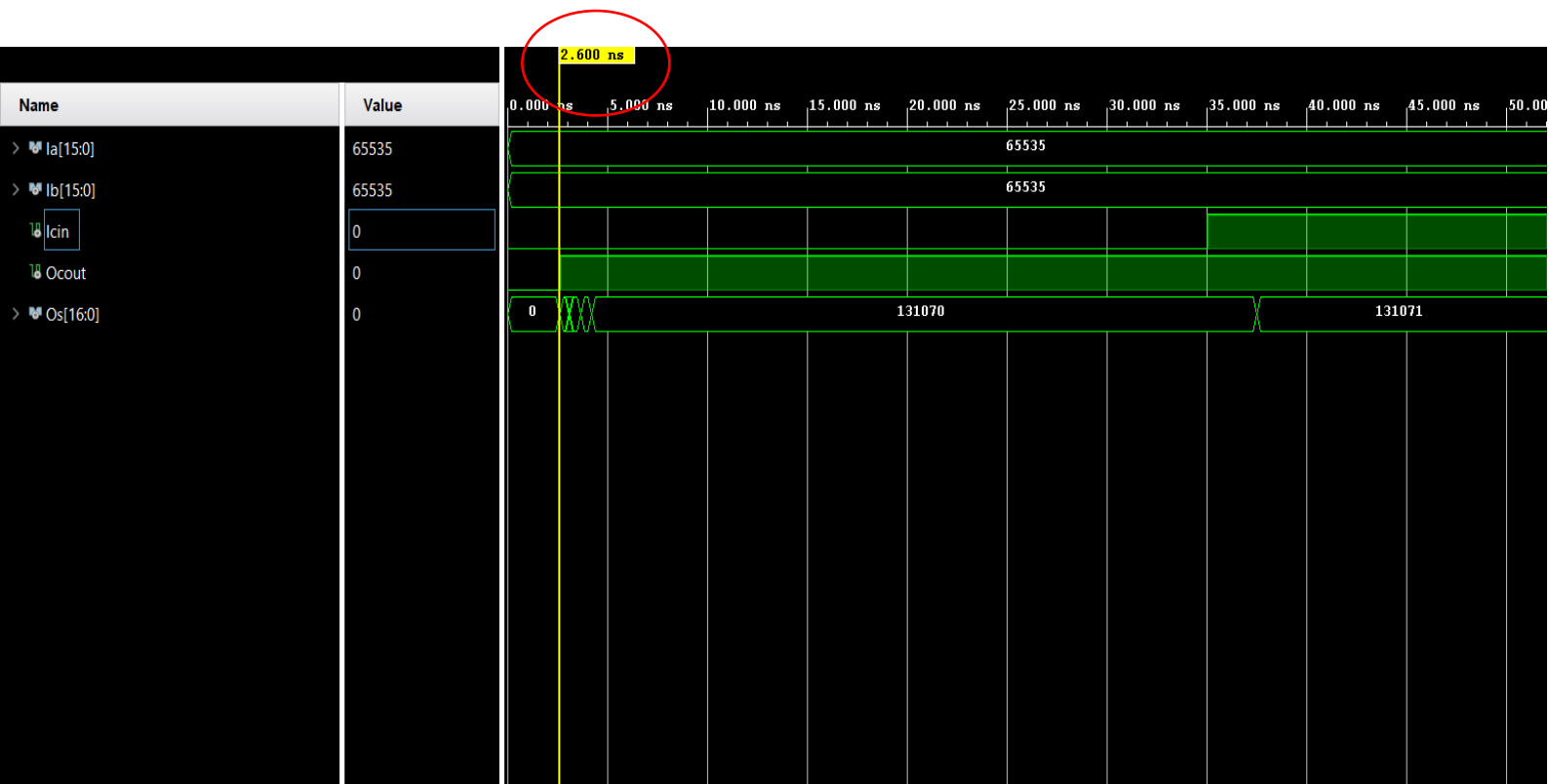
```
32
33     wait for 20 ns;
34     --t=55
35     Ia <= "1111111111111111";
36     Ib <= "0000000000000001";
37     Icin <= '0';
38
39     wait for 10 ns;
40     --t=65
41 end process;
42 end Behavioral;
```



Per completezza di trattazione, oltre che al codice VHDL relativo al testbench, vengono descritti i **diagrammi di timing**, utili a “graficare” l’andamento dei signal nel tempo:



SIMULAZIONE POST-SINTESI



Come si evince dall'immagine sopra riportata, rispetto ai risultati precedentemente ottenuti dalla behavioral simulation, nella simulazione post sintesi, si introduce un ritardo sull'uscita pari a 2.6 ns.

Si allega inoltre il report della sintesi, contenente le risorse utilizzate:

28	1. Slice Logic
29	-----
30	
31	+-----+-----+-----+-----+-----+-----+
32	Site Type Used Fixed Prohibited Available Util%
33	+-----+-----+-----+-----+-----+-----+
34	Slice LUTs* 25 0 0 53200 0.05
35	LUT as Logic 25 0 0 53200 0.05
36	LUT as Memory 0 0 0 17400 0.00
37	Slice Registers 0 0 0 106400 0.00
38	Register as Flip Flop 0 0 0 106400 0.00
39	Register as Latch 0 0 0 106400 0.00
40	F7 Muxes 0 0 0 26600 0.00
41	F8 Muxes 0 0 0 13300 0.00
42	+-----+-----+-----+-----+-----+-----+

OTTIMIZZAZIONE CARRY-SELECT

Come già affermato in precedenza, il Carry-Select a n bit è costituito da p blocchi che elaborano m bit tali che $n = p \times m$.

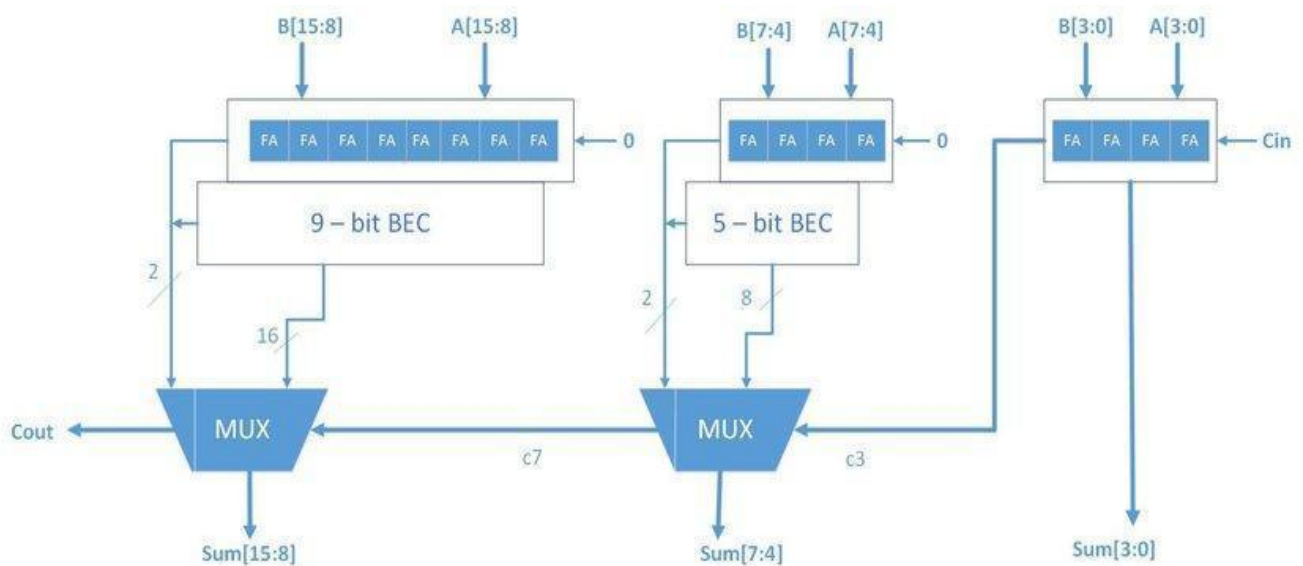
Si indica con T_{FA} il tempo di propagazione di un Full-Adder. Il tempo di propagazione di un RCA a n bit è, nel caso peggiore, $T_{RCA} = nT_{FA}$.

In un Carry-Select le somme calcolate dai vari RCA sono disponibili tutte allo stesso istante $T = mT_{FA}$, mentre i segnali di selezione dei blocchi dal secondo al p -esimo sono disponibili dopo un tempo $T_{CS} = mT_{FA} + qT_{mux}$, con $q \in [1, p - 1]$.

Ad esempio, si supponga per semplicità che $T_{FA} = T_{mux} = 1$.

Riprendendo l'architettura del Carry-Select a 16 bit, è possibile notare che per avere il risultato dell'ultimo blocco occorre un tempo pari a $T_{CS} = 4T_{FA} + 3T_{mux}$. L'ultimo blocco, il quarto, trascorre 4 istanti di tempo a calcolare la somma e per 3 istanti di tempo non fa altro che aspettare. Si potrebbe sfruttare questo tempo aggiungendo nel quarto blocco più Full-Adder rispetto al blocco precedente. Lo stesso vale per il terzo blocco, esso trascorre 2 istanti di tempo ad aspettare, anche qui si potrebbero aggiungere più Full-Adder rispetto al blocco precedente.

È, quindi, possibile migliorare il Carry-Select utilizzando dei blocchi di dimensione variabile, e non più di dimensione fissa m , assegnando un carico computazionale maggiore ai blocchi il cui risultato è richiesto più tardi nel tempo.



1. Design and Characterization of High Speed Carry Select Adder - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/bit-Variable-Modified-Carry-Select-Adder-URCSLA-A-16-bit-carry-select-adder-can-be_fig4_327573343